JAVA COURSE

{ classi e oggetti }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1

INDICE

01 Dai tipi primitivi agli oggetti

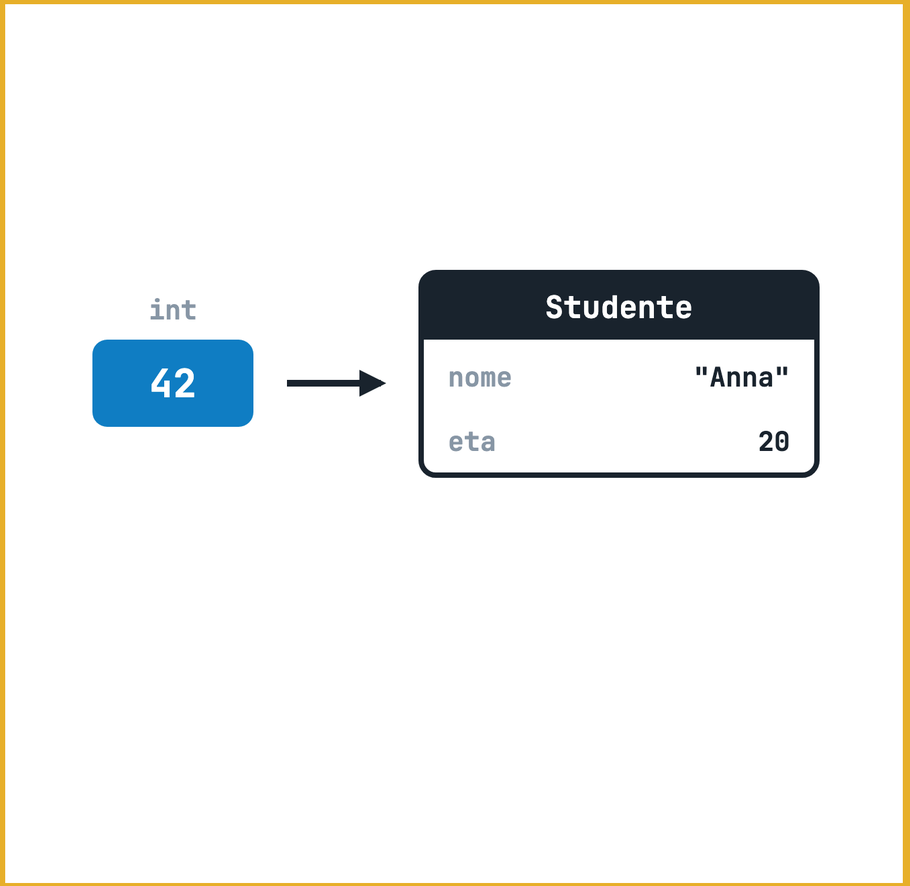
02 Costruttori e metodi

03 Incapsulamento

04 Classi in pratica

01

DAI TIPI PRIMITIVI AGLI OGGETTI



IL PROBLEMA

Per rappresentare uno studente servono più variabili slegate tra loro. Se gli studenti diventano dieci, il codice diventa ingestibile

- *Le variabili che descrivono lo stesso studente non sono collegate*
- *Non si possono passare insieme a un metodo*
- *Serve una struttura che raggruppi i dati di un'entità*

```
String nome1 = "Anna";
int eta1 = 20;
double media1 = 27.5;

String nome2 = "Marco";
int eta2 = 22;
double media2 = 24.0;
// ... e se fossero 30?
```

LA CLASSE: UN PROGETTO

Una **classe** descrive la struttura di un'entità: quali dati contiene e quali operazioni può eseguire

- *La classe è un **modello** (blueprint): definisce campi e metodi*
- `nome`, `eta`, `media` *sono i **campi** (attributi) della classe*
- *La classe da sola non contiene dati: è solo il progetto*

```
class Studente {
    String nome;
    int eta;
    double media;
}
```

L'OGGETTO: UN'ISTANZA

Un **oggetto** è un'istanza della classe: una copia concreta con i propri valori

- `new Studente()` *crea un nuovo oggetto in memoria*
- `s` *è un* ***riferimento*** *all'oggetto appena creato*
- *Si accede ai campi con la notazione* ***punto****:* `s.nome`

```
Studente s = new Studente();
s.nome = "Anna";
s.eta = 20;
s.media = 27.5;
```

CLASSE VS OGGETTO

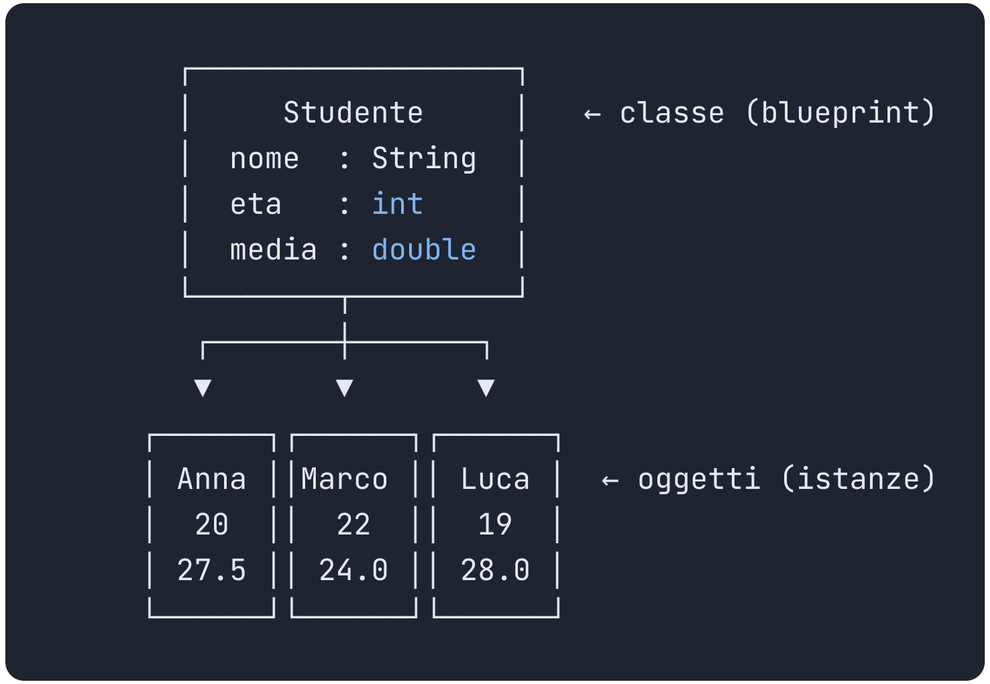
La classe è il **progetto della casa**, l'oggetto è la **casa costruita**

Classe	Oggetto
Definizione astratta	Istanza concreta
Una sola per tipo	Quante se ne vogliono
Descrive campi e metodi	Contiene valori specifici
Esiste nel codice sorgente	Esiste in memoria a runtime
`class Studente { ... }`	`new Studente()`

DALLA CLASSE AGLI OGGETTI

Da un'unica classe si possono creare più oggetti indipendenti

- *Ogni oggetto ha la stessa struttura ma valori diversi*
- *Modificare un oggetto non influenza gli altri*



CREARE E USARE OGGETTI

Esempio completo: definire la classe, creare due oggetti, leggere i loro campi

```
class Studente {
    String nome;
    int eta;
    double media;
}
```

```
public class Main {
    public static void main(String[] args) {
        Studente s1 = new Studente();
        s1.nome = "Anna";
        s1.eta = 20;

        Studente s2 = new Studente();
        s2.nome = "Marco";
        s2.eta = 22;

        System.out.println(s1.nome + " ha " + s1.eta + " anni");
        System.out.println(s2.nome + " ha " + s2.eta + " anni");
    }
}
```

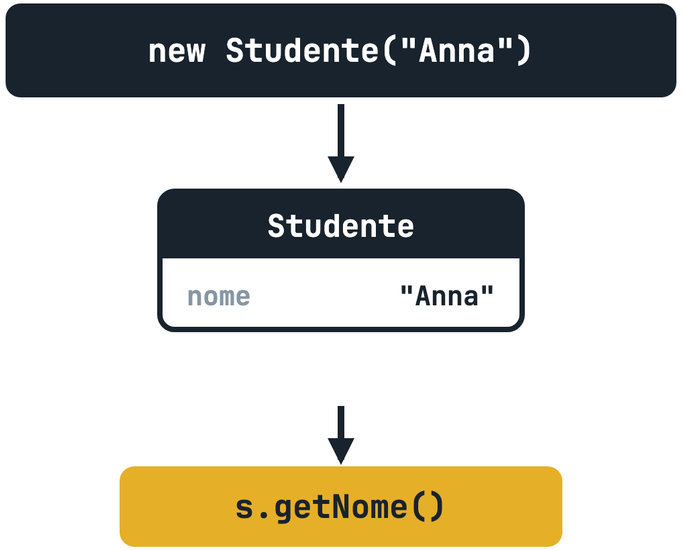
I VALORI PREDEFINITI DEI CAMPI

Quando si crea un oggetto con `new`, ogni campo riceve un valore predefinito

- *Stessa regola degli array: i campi non inizializzati hanno un valore di default*
- *Le variabili locali invece non ce l'hanno: usarle senza assegnarle è un errore di compilazione*

Tipo del campo	Valore predefinito
`int`, `long`, `short`, `byte`	`0`
`double`, `float`	`0.0`
`boolean`	`false`
`char`	`'\u0000'`
`String` (e altri oggetti)	`null`

02
COSTRUTTORI E METODI



IL COSTRUTTORE

Un **costruttore** è un blocco simile a un metodo che inizializza l'oggetto al momento della creazione

- *Ha lo **stesso nome** della classe*
- ***Non ha tipo di ritorno** (nemmeno `void`)*
- *Viene chiamato automaticamente con `new`*

```
class Studente {
    String nome;
    int eta;

    // costruttore
    Studente(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }
}

// uso
Studente s = new Studente("Anna", 20);
```

THIS

`this` è il riferimento all'oggetto corrente. Serve per distinguere i campi dai parametri quando hanno lo stesso nome

- `this.nome` *si riferisce al* ***campo*** *dell'oggetto*
- `nome` *(senza* `this` *) si riferisce al* ***parametro*** *del costruttore*
- *Senza* `this` *, il parametro "nasconde" il campo (shadowing)*
- *Buona pratica: usare sempre* `this` *nel costruttore*

```
Studente(String nome, int eta) {
    this.nome = nome;   // campo ← parametro
    this.eta = eta;      // campo ← parametro
}
```

COSTRUTTORE DI DEFAULT

Se non si scrive nessun costruttore, Java ne fornisce uno **vuoto** automaticamente

- *Appena si definisce un costruttore, quello di default **sparisce***
- *Per averlo di nuovo bisogna scriverlo esplicitamente*

```
// nessun costruttore scritto
class Punto {
    int x;
    int y;
}
Punto p = new Punto();   // OK: costruttore di default

// se si scrive un costruttore con parametri...
class Punto {
    int x, y;
    Punto(int x, int y) { this.x = x; this.y = y; }
}
Punto p = new Punto();   // ERRORE! Non esiste più
```

PIÙ COSTRUTTORI

Una classe può avere più costruttori con parametri diversi (overloading)

```java
class Studente {
    String nome;
    int eta;

    Studente() {
        this.nome = "Sconosciuto";
        this.eta = 0;
    }

    Studente(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }
}
```

```java
// costruttore vuoto
Studente s1 = new Studente();

// costruttore con parametri
Studente s2 = new Studente("Anna", 20);
```

OVERLOADING DEI METODI

Come i costruttori, anche i metodi possono avere lo stesso nome con parametri diversi: è l'**overloading**

- *Il compilatore sceglie il metodo dal numero e dal tipo degli argomenti*
- *Il tipo di ritorno da solo non basta: due metodi che differiscono solo per quello non compilano*

```
static int somma(int a, int b) {
    return a + b;
}
static int somma(int a, int b, int c) {
    return a + b + c;
}
static double somma(double a, double b) {
    return a + b;
}

somma(3, 5);       // 8
somma(1, 2, 3);    // 6
somma(1.5, 2.5);   // 4.0
```

I METODI DI ISTANZA

Un metodo di istanza opera sui campi dell'oggetto. Non ha la parola chiave `static`

- *Il metodo accede ai campi dell'oggetto su cui viene chiamato*
- *Si invoca con la notazione punto:* `oggetto.metodo()`

```
class Studente {
    String nome;
    int eta;

    Studente(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }

    void saluta() {
        System.out.println("Ciao, sono " + nome);
    }
}

Studente s = new Studente("Anna", 20);
s.saluta();   // "Ciao, sono Anna"
```

METODI CON PARAMETRI E RITORNO

I metodi di istanza possono avere parametri e restituire valori, come i metodi statici

- *La differenza con i metodi statici: qui* `this` *esiste*
- *Il metodo opera sui dati dell'oggetto specifico*

```
class Studente {
    String nome;
    int eta;
    double media;

    // metodo con parametro
    void aggiornaMedia(double nuovaMedia) {
        this.media = nuovaMedia;
    }

    // metodo con valore di ritorno
    boolean isSufficiente() {
        return media >= 18.0;
    }
}
```

TOSTRING()

`toString()` restituisce una rappresentazione testuale dell'oggetto. Viene chiamato automaticamente da `println`

- *Senza* `toString()`, `println` *stampa un codice come* `Studente@1b6d3586`

```
class Studente {
    String nome;
    int eta;

    Studente(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }

    public String toString() {
        return "Studente{nome=" + nome + ", eta=" + eta + "}";
    }
}
```

```
Studente s = new Studente("Anna", 20);
System.out.println(s);
// Studente{nome=Anna, eta=20}
```

ESEMPIO: CONTOCORRENTE

Una classe più completa: un conto corrente con saldo, versamenti e prelievi

```
class ContoCorrente {
    String titolare;
    double saldo;

    ContoCorrente(String titolare, double saldo) {
        this.titolare = titolare;
        this.saldo = saldo;
    }
    void versa(double importo) {
        saldo += importo;
    }
    void preleva(double importo) {
        if (importo <= saldo) {
            saldo -= importo;
        } else {
            System.out.println("Saldo insufficiente");
        }
    }
    public String toString() {
        return titolare + " — saldo: " + saldo + " EUR";
    }
}
```

USARE CONTOCORRENTE

Creare un conto, versare e prelevare denaro

- *Ogni operazione modifica lo **stato** dell'oggetto*
- *Il metodo* `preleva` *contiene una **logica di protezione***

```
ContoCorrente c = new ContoCorrente("Anna", 1000);
System.out.println(c);        // Anna — saldo: 1000.0 EUR

c.versa(500);
System.out.println(c);        // Anna — saldo: 1500.0 EUR

c.preleva(200);
System.out.println(c);        // Anna — saldo: 1300.0 EUR

c.preleva(5000);              // Saldo insufficiente
```

PIÙ OGGETTI, STESSA CLASSE

Ogni istanza ha i propri valori. Le operazioni su un oggetto non influenzano gli altri

- `c1` e `c2` *sono due oggetti separati in memoria*
- *Versare su* `c1` *non cambia il saldo di* `c2`

```
ContoCorrente c1 = new ContoCorrente("Anna", 1000);
ContoCorrente c2 = new ContoCorrente("Marco", 500);

c1.versa(200);

System.out.println(c1);  // Anna — saldo: 1200.0 EUR
System.out.println(c2);  // Marco — saldo: 500.0 EUR
```

NULL

`null` indica che un riferimento **non punta a nessun oggetto**

- *Accedere a un campo o metodo su* `null` *causa* `NullPointerException`
- *È l'errore più comune in Java*
- *Prima di usare un riferimento, verificare che non sia* `null`

```
Studente s = null;   // nessun oggetto

System.out.println(s.nome);  // ERRORE a runtime!
```

Terminale

```
Exception in thread "main"
  java.lang.NullPointerException:
  Cannot read field "nome"
  because "s" is null
```

QUIZ

Cosa stampa il seguente frammento?

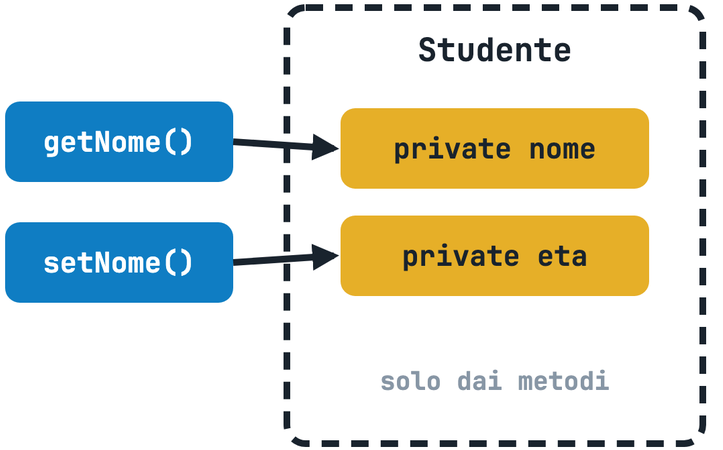
```
class Contatore {
    int valore;
    Contatore(int valore) { this.valore = valore; }
    void incrementa() { valore++; }
    public String toString() { return "Contatore: " + valore; }
}

Contatore a = new Contatore(5);
Contatore b = new Contatore(10);
a.incrementa();
a.incrementa();
b.incrementa();
System.out.println(a);
System.out.println(b);
```

RISPOSTE

- `a` *parte da 5, viene incrementato due volte:* ***Contatore: 7***
- `b` *parte da 10, viene incrementato una volta:* ***Contatore: 11***
- *Ogni oggetto ha il proprio campo* `valore`*, indipendente*
- `toString()` *viene chiamato automaticamente da* `println`

03
INCAPSULAMENTO



IL PROBLEMA

Se i campi sono accessibili dall'esterno, chiunque può assegnare valori incoerenti

- *Il saldo diventa negativo senza passare da* `preleva()`
- *Si perde la validazione dei dati*
- *Il codice esterno può "rompere" lo stato dell'oggetto*
- *Serve un meccanismo per* ***proteggere*** *i campi*

```
ContoCorrente c = new ContoCorrente("Anna", 1000);
c.saldo = -999999;   // nessuno lo impedisce!
```

PRIVATE

La parola chiave `private` rende un campo accessibile **solo dall'interno** della classe

- `private` *blocca l'accesso diretto dall'esterno della classe*
- *I metodi interni della classe possono ancora usare il campo*

```
class ContoCorrente {
    private String titolare;
    private double saldo;

    ContoCorrente(String titolare, double saldo) {
        this.titolare = titolare;
        this.saldo = saldo;
    }
}

ContoCorrente c = new ContoCorrente("Anna", 1000);
c.saldo = -999999;   // ERRORE di compilazione!
```

GETTER E SETTER

Per leggere e modificare i campi privati si usano metodi **pubblici** dedicati

```
class ContoCorrente {
    private String titolare;
    private double saldo;

    // getter
    public String getTitolare() {
        return titolare;
    }

    // getter
    public double getSaldo() {
        return saldo;
    }

    // setter con validazione
    public void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        }
    }
}
```

PERCHÉ INCAPSULARE

L'incapsulamento protegge i dati e garantisce la coerenza dell'oggetto

Vantaggio	Senza incapsulamento	Con incapsulamento
Validazione	Chiunque assegna qualsiasi valore	Il setter controlla i dati
Coerenza	Lo stato può diventare invalido	Lo stato resta sempre valido
Flessibilità	Cambiare l'implementazione rompe il codice esterno	L'implementazione interna può cambiare liberamente
Controllo	Nessuno sa chi modifica cosa	Ogni modifica passa per un metodo

VALIDAZIONE NEL SETTER

Il setter è il punto ideale per validare i dati prima di assegnarli

- *Il campo rimane protetto da valori assurdi*
- *La logica di validazione è in un unico punto*

```
public void setEta(int eta) {
    if (eta >= 0 && eta <= 130) {
        this.eta = eta;
    } else {
        System.out.println("Età non valida: " + eta);
    }
}

s.setEta(25);    // OK
s.setEta(-5);    // "Età non valida: -5"
s.setEta(200);   // "Età non valida: 200"
```

LA CONVENZIONE JAVABEAN

I nomi di getter e setter seguono una convenzione precisa

- `get` + *nome del campo con iniziale maiuscola*
- *Per i* `boolean` *il getter usa* `is` *invece di* `get`
- *Questa convenzione è usata ovunque in Java*

Campo	Getter	Setter
`nome`	`getNome()`	`setNome(String)`
`eta`	`getEta()`	`setEta(int)`
`saldo`	`getSaldo()`	`setSaldo(double)`
`attivo`	`isAttivo()`	`setAttivo(boolean)`

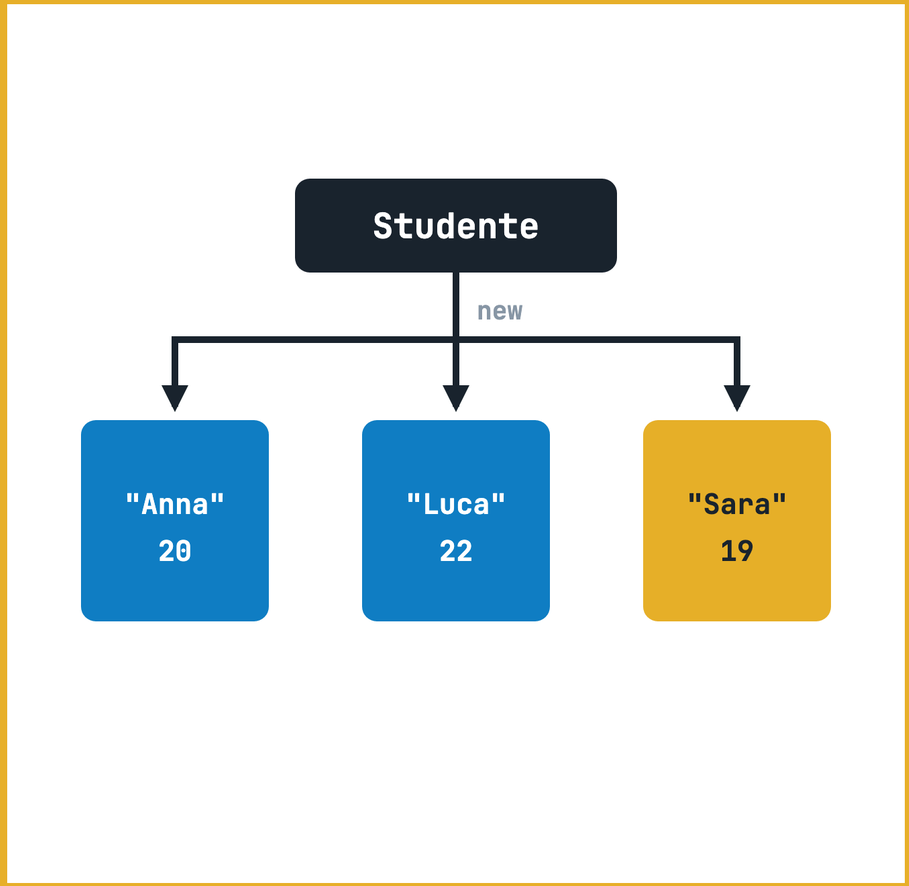
PUBLIC E PRIVATE

I modificatori di accesso decidono chi può usare un campo o un metodo

- *Regola d'oro: **campi privati, metodi pubblici***
- *I metodi pubblici sono l'**interfaccia** della classe*
- *L'implementazione interna resta nascosta*

Modificatore	Visibilità	Uso tipico
`private`	Solo dentro la classe	Campi
`public`	Ovunque	Metodi, costruttori
nessuno	Classi della stessa cartella (package)	Quello usato finora negli esempi

04
CLASSI IN PRATICA



UN FILE PER CLASSE

In Java ogni classe pubblica va in un file con lo stesso nome

- `Studente.java` *contiene la classe* `Studente`
- `Main.java` *contiene il* `main` *che usa la classe*
- *Si compilano insieme:*
  `javac Studente.java Main.java`

```
// file: Studente.java
public class Studente {
    private String nome;
    private int eta;
    // costruttore, getter, setter, toString...
}

// file: Main.java
public class Main {
    public static void main(String[] args) {
        Studente s = new Studente("Anna", 20);
        System.out.println(s);
    }
}
```

ARRAY DI OGGETTI

Si possono creare array i cui elementi sono oggetti

- `new Studente[3]` *crea l'array con 3 celle, tutte a* `null`
- *Ogni cella va riempita con* `new Studente(...)`

```
Studente[] classe = new Studente[3];

classe[0] = new Studente("Anna", 20);
classe[1] = new Studente("Marco", 22);
classe[2] = new Studente("Luca", 19);

for (Studente s : classe) {
    System.out.println(s);
}
```

Terminale

```
Studente{nome=Anna, eta=20}
Studente{nome=Marco, eta=22}
Studente{nome=Luca, eta=19}
```

ARRAY DI OGGETTI E NULL

Creare l'array non crea gli oggetti. Le celle partono a `null`

- `new Studente[3]` *alloca solo lo spazio per 3 riferimenti*
- *Bisogna creare ogni oggetto singolarmente con* `new`
- *Prima di usare un elemento, verificare che non sia* `null`

```
Studente[] classe = new Studente[3];
// classe[0] è null, classe[1] è null, classe[2] è null

System.out.println(classe[0].nome);  // NullPointerException!
```

PASSARE OGGETTI AI METODI

Quando si passa un oggetto a un metodo, si passa il **riferimento**: il metodo lavora sull'originale

- *Come per gli array: si copia il riferimento, non l'oggetto*
- *Le modifiche dentro il metodo si riflettono sull'originale*
- *I tipi primitivi (`int`, `double`) vengono copiati per valore*

```
static void promuovi(Studente s) {
    s.setEta(s.getEta() + 1);   // modifica l'originale!
}

public static void main(String[] args) {
    Studente anna = new Studente("Anna", 20);
    promuovi(anna);
    System.out.println(anna.getEta());  // 21
}
```

STATIC VS ISTANZA

Ora `static` ha un significato preciso: appartiene alla classe, non all'oggetto

- `static` = "*non serve un'istanza per chiamarlo*"
- *Finora abbiamo usato solo metodi* `static` *perché non avevamo oggetti*

Aspetto	Metodo `static`	Metodo di istanza
Appartiene a	La classe	L'oggetto
Accede ai campi	No (solo parametri)	Sì (tramite `this`)
Chiamata	`Math.sqrt(4)`	`s.saluta()`
Serve un oggetto	No	Sì
Uso tipico	Utilità, `main`	Comportamento dell'oggetto

RICORSIONE

Un metodo **ricorsivo** chiama se stesso su un problema più piccolo, finché arriva a un **caso base**

- *Il caso base ferma le chiamate: senza, il metodo non finisce mai*
- *Ogni chiamata aspetta il risultato di quella successiva*

```
static long fattoriale(int n) {
    if (n <= 1) {
        return 1;                    // caso base
    }
    return n * fattoriale(n - 1);  // chiamata ricorsiva
}

fattoriale(4);
// 4 * fattoriale(3)
// 4 * 3 * fattoriale(2)
// 4 * 3 * 2 * fattoriale(1)
// 4 * 3 * 2 * 1 = 24
```

SENZA CASO BASE

Se manca il caso base, le chiamate si accumulano finché la memoria riservata alle chiamate finisce

- *Ogni chiamata occupa spazio nello **stack** fino a quando non restituisce*
- *Una ricorsione si può sempre riscrivere con un ciclo*

```
static long fattoriale(int n) {
    return n * fattoriale(n - 1);
}
```

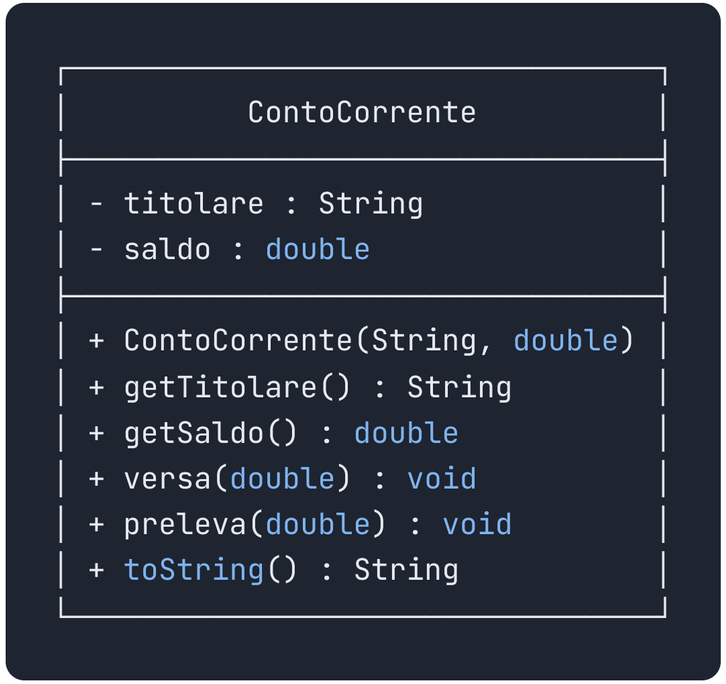
Terminale

```
Exception in thread "main"
  java.lang.StackOverflowError
```

UML SEMPLIFICATO

Un diagramma UML rappresenta una classe con tre sezioni: nome, campi, metodi

- `-` *indica* `private`, `+` *indica* `public`
- *Utile per progettare una classe prima di scrivere il codice*



ERRORI COMUNI

I bug più frequenti quando si inizia a lavorare con le classi

Errore	Causa	Correzione
`NullPointerException`	Usare un riferimento `null`	Creare l'oggetto con `new`
Campo non modificato dal costruttore	Dimenticato `this`	`this.nome = nome`
Costruttore di default mancante	Definito un altro costruttore	Aggiungere il costruttore vuoto
Accesso a campo `private`	Accesso diretto dall'esterno	Usare getter/setter
`static` su metodo di istanza	Confusione tra classe e oggetto	Togliere `static`
Due `String` confrontate con `==`	Confronta i riferimenti, non il testo	Usare `.equals()`

QUIZ

Cosa stampa il seguente frammento?

```
class Punto {
    private int x, y;
    Punto(int x, int y) { this.x = x; this.y = y; }
    public int getX() { return x; }
    public int getY() { return y; }
}

Punto[] punti = new Punto[3];
punti[0] = new Punto(1, 2);
punti[1] = new Punto(3, 4);

System.out.println(punti[0].getX());
System.out.println(punti[1].getY());
System.out.println(punti[2]);
```

RISPOSTE

- `punti[0].getX()` — *il punto (1,2), restituisce x:* ***1***
- `punti[1].getY()` — *il punto (3,4), restituisce y:* ***4***
- `punti[2]` — *non è stato creato nessun oggetto:* ***null***
- *Se si facesse* `punti[2].getX()` *si otterrebbe* `NullPointerException`

RIEPILOGO

- ***Classe*** *— progetto con campi e metodi*
- ***Oggetto*** *— istanza creata con* `new`
- ***Costruttore*** *— inizializza l'oggetto*
- `this` *— l'oggetto corrente*
- ***Metodi di istanza*** *— senza* `static`
- ***Overloading*** *— stesso nome, altri parametri*
- ***Ricorsione*** *— un metodo che chiama se stesso*
- ***Incapsulamento*** *— campi* `private`*, getter e setter*
- `toString()` *— l'oggetto come testo*
- `null` *— nessun oggetto*
- ***Array di oggetti*** *— celle a* `null`

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1